



terv▶▶

practical learning redefined

One point solution for all your placement needs



OOPs Concepts

- Object-Oriented Programming
 - ✓ Object
 - ✓ Class
 - ✓ Inheritance
 - ✓ Polymorphism
 - ✓ Abstraction
 - ✓ Encapsulation

- **Object** – Any entity that has state and behaviour
- **Class** – Collection of objects
- **Inheritance** – When one object acquires all the properties and behaviours of parent object
- **Polymorphism** – When one task is performed by different ways
- **Abstraction** – Hiding internal details and showing functionality
- **Encapsulation** – Binding (or wrapping) code and data together into a single unit

Object

- state
- Behaviour
- Identity

A class in Java can contain:

- fields
- methods
- constructors
- blocks
- nested class and interface

new keyword – Used to allocate memory at run time.
All objects get memory in Heap memory area

Naming conventions



- **Naming convention** is a rule to follow as you decide what to name your identifiers such as class, package, variable, constant, method etc.

Name	Convention
class name	Start with uppercase letter. Ex: String, Color, Button, System
interface name	Start with uppercase letter. Ex: Runnable, Remote
method name	Start with lowercase letter. Ex: main(), print(), println()
variable name	Start with lowercase letter. Ex: firstName, orderNumber
package name	In lowercase letter Ex: java, lang, sql, util
constants name	In uppercase letter. Ex: RED, YELLOW

Ways to initialize object:

- By reference variable – Initializing object simply means storing data into object
- By method
- By constructor

Object and Class Example: main within class



```
public class Oops {  
    //Instance variable (Inside class,Outside Method)  
    int roll;  
    String name;  
    public static void main(String[] args) {  
        Oops obj = new Oops(); //obj is object, Oops is class  
        obj.roll = 10;  
        obj.name = "Terv";  
        System.out.println(obj.roll+" "+obj.name);  
    }  
}
```

Object and Class Example: main outside class



```
class Main{  
    //Instance variable  
    (Inside class,  
    Outside Method)  
    int roll;  
    String name;  
}
```

```
public class Oops {  
    public static void main(String[]  
    args) {  
        Main obj = new Main();  
        obj.roll = 20;  
        obj.name = "Terv";  
        System.out.println(obj.roll+"  
        "+obj.name);  
    }  
}
```

Initialization through reference:



```
class Main{  
  //Instance variable  
  (Inside class,Outside  
  Method)  
  int roll;  
  String name;  
}
```

```
public class Oops {  
  //Create Multiple Objects  
  public static void main(String[] args) {  
    Main obj = new Main();  
    Main obj1 = new Main();  
    obj.roll = 20;  
    obj.name = "Terv";  
    obj1.roll = 30;  
    obj1.name = "Avis";  
    System.out.println(obj.roll+"  
    "+obj.name);  
    System.out.println(obj1.roll+"  
    "+obj1.name);  
  }  
}
```

Initialization through method:



```
class Main{
    int roll;
    String name;
    void input(int a,String b){
        roll = a;
        name = b;
    }
}

public class Oops {
    //Create Multiple Objects
    public static void main(String[] args) {
        Main obj = new Main();
        obj.input(2143, "Terv");
        System.out.println(obj.roll+" "+obj.name);
    }
}
```

Initialization through constructor:



```
class Main{
int roll;
String name;
Main(int roll,String name){
this.roll = roll;
this.name = name;
}//this keyword to access instance var within method
}

public class Oops {
//Create Multiple Objects
public static void main(String[] args) {
Main obj = new Main(2131,"Terv");
System.out.println(obj.roll+" "+obj.name);
}
}
```

Anonymous object:

→ If you have to use an object only once, anonymous object is a good approach

→ **new** Student();

Calling method through reference:

```
Student s = new Student();  
s.scores(5);
```

Calling method through anonymous object

```
new Student().scores(5);
```

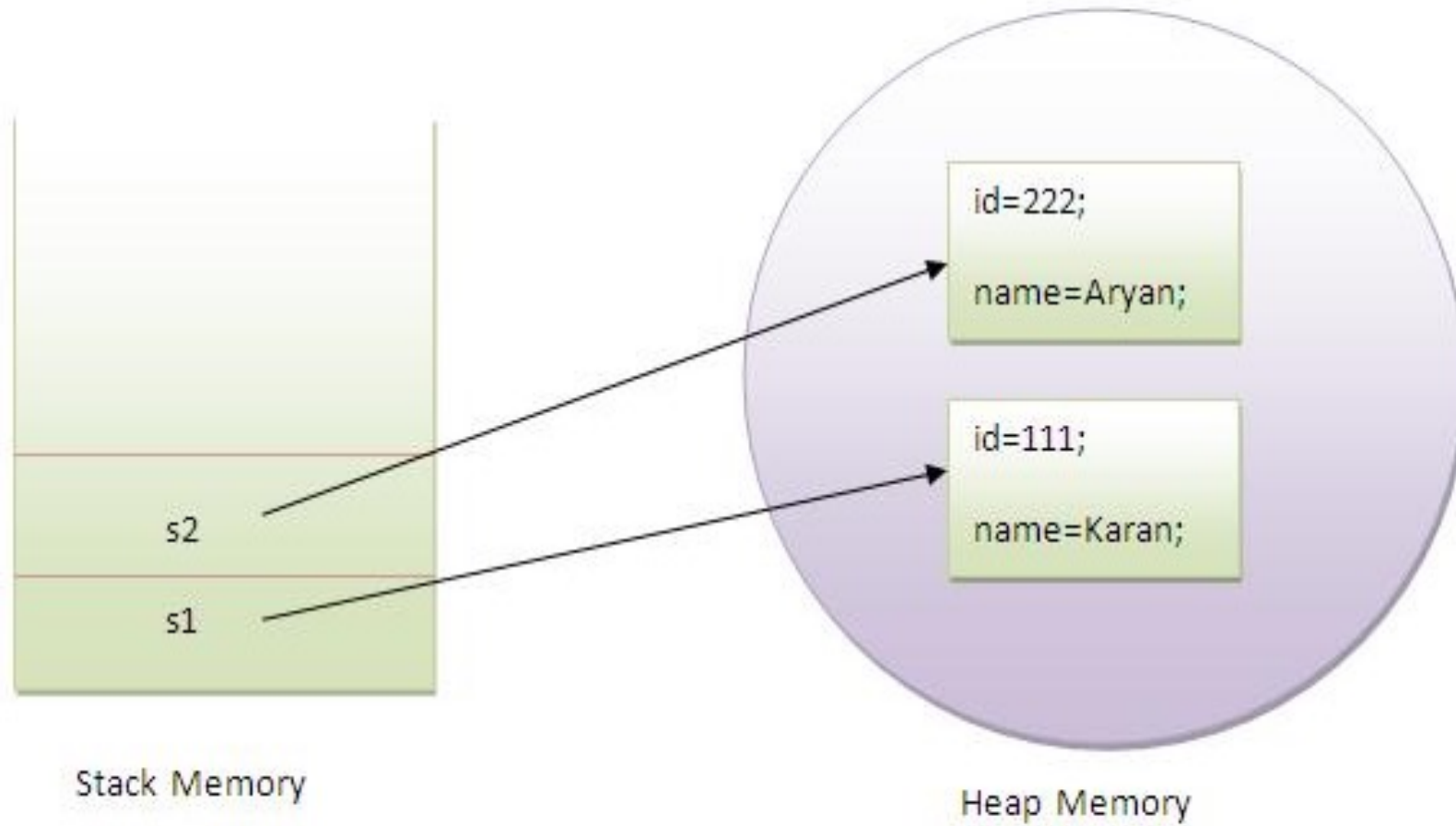
Anonymous object example:



```
class Main{
int roll;
String name;
Main(int roll,String name){
this.roll = roll;
this.name = name;
} // this keyword to access instance var within method
}

public class Oops {
public static void main(String[] args) {
System.out.println(new Main(2131,"Terv").roll+" "+
new Main(2131,"Team Terv").name);
}
}
```

Memory Allocation of Objects



Inheritance

The process where one class acquires the properties (methods and fields) of another class.

- Subclass – which inherits the properties of another class.
- Superclass – whose properties are inherited

Inheritance represents the **IS-A relationship**, also known as parent-child relationship.

SYNTAX:

```
class Subclass_name extends Superclass_name
{
    //methods and fields
}
```

extends is the keyword used to inherit the properties of a class

Example:

```
class Employee
```

```
{
    float salary;
}
```

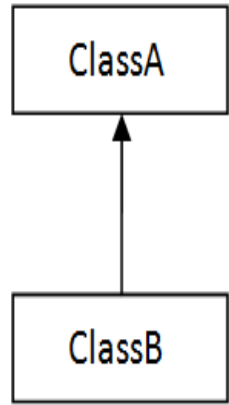
```
class Programmer extends Employee
```

```
{
    int bonus;
    public static void main(String args[])
    {
        Programmer p=new Programmer();
        p.salary = 40000;
        p.bonus = 10000;
        System.out.println("Programmer salary is:"+p.salary);
        System.out.println("Bonus of Programmer is:"+p.bonus);
    }
}
```

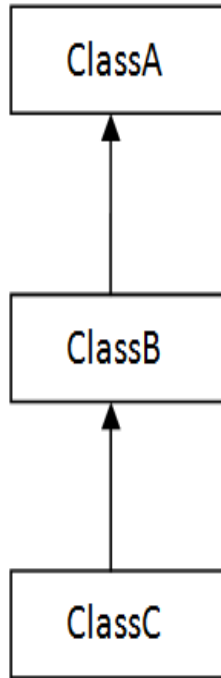
Output:

```
Programmer salary is:40000.0
Bonus of programmer is:10000
```

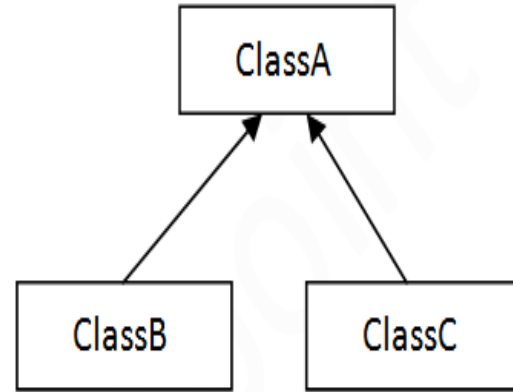
Types of inheritance



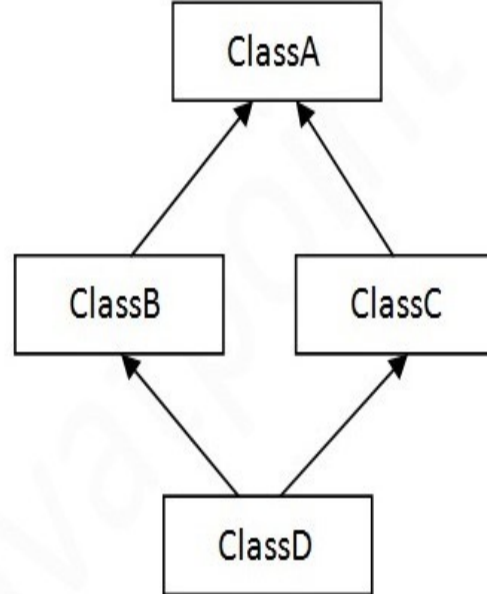
1) Single



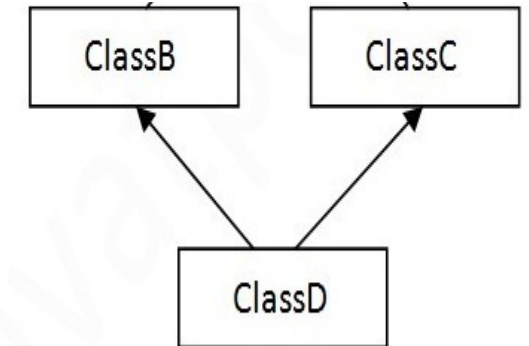
2) Multilevel



3) Hierarchical



4) Hybrid



5) Multiple

Without using inheritance

```
class Parent{
void car() {
System.out.println("BMW");
}
void TV() {
System.out.println("SONY");
}
}
class Child{
void toys() {
System.out.println("Remote
car");
}
}
```

```
public class Oops {
public static void
main(String[] args) {
Child obj = new Child();
obj.toys();
obj.car();
obj.TV();
}
}
```

OP: Exception in thread "main"
java.lang.Error: Unresolved compilation
problems:
The method car() is undefined for the type
Child
The method TV() is undefined for the type
Child

Single inheritance :



```
class Parent{
void car() {
System.out.println("BMW");
}
void TV() {
System.out.println("SONY");
}
}
class Child extends Parent{
void toys() {
System.out.println("Remote
car");
}
}
```

```
public class Oops {
public static void
main(String[] args) {
Child obj = new Child();
obj.toys();
obj.car();
obj.TV();
}
}
```

OP:

Remote car
BMW
SONY

Multilevel inheritance



```
class Grandpa{
void land() {
System.out.println("Resort");
}
}
class Parent extends Grandpa{
void car() {
System.out.println("RR");
}
void bike() {
System.out.println("Jawa");
}
}
class Child extends Parent{
void toys() {
System.out.println("Bey_Blade");
System.out.println("PS-5");
}
}
```

```
class info{
public static void main(String
[]arg) {
Child obj1 = new Child();
Parent obj2 = new Parent();
Grandpa obj3 = new Grandpa();
System.out.println("Child:");
obj1.bike(); obj1.car();
obj1.land(); obj1.toys();
System.out.println("Parent:");
obj2.bike(); obj2.car();
obj2.land();
System.out.println("Grandpa:");
obj3.land();
} //End of main
} //End of class
```

Hierarchical inheritance



```
class Parent{
void car() {
System.out.println("RR");
}
void bike() {
System.out.println("Jawa");
}
class Brother extends Parent{
void toys() {
System.out.println("Bey_Blade");
System.out.println("PS-5");
}}
class Sister extends Parent{
void cosmetics() {
System.out.println("Lipstick");
System.out.println("Eye_liner");
}}
```

```
class info{
public static void main(String []arg)
{
Brother obj1 = new Brother();
Sister obj2 = new Sister();
Parent obj3 = new Parent();
System.out.println("Brother:");
obj1.bike(); obj1.car(); obj1.toys();
System.out.println("Sister:");
obj2.bike(); obj2.car();
obj2.cosmetics();
//obj1.cosmetics(); // No Access
//obj2.toys(); // No Access
//obj3.toys(); obj3.cosmetics()
} //End of main
} //End of class
```

Hybrid inheritance



```
public class Main{
public static void main (String[] args) {
    grandchild o=new grandchild();
        o.method1();    o.method2();    o.method3();
        arun o1=new arun();
o1.method4();
venkat o2=new venkat();
    o2.method5();    }}
class parent{
    void method1(){
        System.out.println("parent class");}}
class child extends parent {
    void method2(){
        System.out.println("child class"); }}
```

```
class grandchild extends child {
    void method3(){
System.out.println("grandchildclass");
}}
class arun extends parent{
    void method4(){
        System.out.println("arun class");    }}
class hari extends parent{
    void method5(){
        System.out.println("hari class");
    } }
```

Polymorphism

- Polymorphism means many forms.

Achieved in two ways:

- Compile time polymorphism – Method Overloading
- Runtime polymorphism – Method Overriding

Method Overloading

- If a class have multiple methods by same name but different parameters, it is known as **Method Overloading**
- Increases the readability of the program
- **Two ways :**
 - By changing number of arguments
 - By changing the data type

Changing the number of arguments:

```
public class Oops {  
    static void selvam(int a,int b) {  
        System.out.println(a+b+" Two arg");  
    }  
    static void selvam(int a,int b,int c) {  
        System.out.println(a+b-c+" Three arg");  
    }  
    public static void main(String[] args) {  
        selvam(10,20);// two arg mthd  
        selvam(10,50,20);// three arg mthd  
    }  
}
```

OP:

30 Two arg
40 Three arg

Changing datatype of arguments:

```
public class Oops {
    static void selvam(int a,int b) {
        System.out.println(a+b+" int int arg");
    }
}
```

```
static void selvam(double a,int b) {
    System.out.println(a+b+" double int arg");
}
```

```
public static void main(String[] args) {
    selvam(10,20); // two arg mthd
    selvam(10.0,50); // three arg mthd
}
```

OP:

30 Two arg

60.0 Three arg

Can we overload main() method?



```
public class Oops {  
    static void main(String arg) {  
        System.out.println(arg);  
    }  
    public static void main(String[] args) {  
        System.out.println(args);  
        main("Team Terv");  
    }  
}
```

Method Overriding:

- If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in java**.
- Method overriding is used for runtime polymorphism

Rules for Java Method Overriding

- must be IS-A relationship (inheritance).
- method must have same name as in the parent class
- method must have same parameter as in the parent class.

Example:

```
class Parent{
void chocolate() {
System.out.println("Kit-Kat");
}
}

class Child extends Parent{
void chocolate() {
System.out.println
("Dairy Milk Silk");
}
}
```

```
class info{
public static void main(String
[]arg) {
Child obj = new Child();
obj.chocolate();
Parent obj2 = new Parent();
obj2.chocolate();
} //End of main
} //End of class
```

OP:

Dairy Milk Silk
Kit-Kat

Example: With **super** Key word

```
class Parent{
void chocolate() {
System.out.println
("Kit-Kat");
}
}

class Child extends Parent{
void chocolate() {
super.chocolate();
System.out.println
("Dairy Milk Silk");
}}
```

```
class info{
public static void main(String
[]arg) {
Child obj = new Child();
obj.chocolate();
Parent obj2 = new Parent();
obj2.chocolate();
} //End of main
} //End of class
```

OP:
Kit-Kat
Dairy Milk Silk
Kit-Kat

Abstraction:

- It is a process of hiding the implementation details and showing only functionality to the user.
- Abstraction lets you focus on what the object does instead of how it does it.

Ways to achieve Abstraction

- Abstract class
- Interface

Abstract Class:

- An abstract class can not be **instantiated** (you are not allowed to create **object** of Abstract class)
- Both concrete methods and abstract methods can be included
- It does not provide 100% abstraction (Partial Abstraction- because it also includes concrete methods)
- **Remember two rules:**
 - 1) If the class is having few abstract methods and few concrete methods: declare it as abstract class.
 - 2) If the class is having only abstract methods: declare it as interface.

Creating object for Abstract Class..!!

```
abstract class Bike
{
    abstract void run();
}
class Honda4 extends Bike
{
    void run(){System.out.println("running safely..");}
    public static void main(String args[]){
        Bike obj = new Bike();
        obj.run();
    }
}
```

Error:

Unresolved compilation problem: Cannot instantiate the type Bike

Abstract class example:

```
abstract class shapes{
    abstract void square();
    abstract void rectangle();
    abstract void circle();
}
class Draw_pencil extends shapes{
    void square(){
        System.out.println
        ("Square Drawing using pencil");
    }
    void rectangle() {
        System.out.println
        ("Rectangle Drawing using pencil");
    }
    void circle() {
        System.out.println
        ("Circle Drawing using pencil");
    }
}
```

```
class Draw_pen extends shapes{
    void square(){
        System.out.println
        ("Square Drawing using pen");
    }
    void rectangle() {
        System.out.println
        ("Rectangle Drawing using pen");
    }
    void circle() {
        System.out.println
        ("Circle Drawing using pen");
    }
}
```

Abstract class example:

```
public class Ops {
    public static void main(String[] arg) {
        shapes obj1 = new Draw_pencil();
        System.out.println(obj1);
        obj1.square();
        shapes obj2 = new Draw_pen();
        System.out.println(obj2);
        obj2.square();
    }
}
```

OP:

```
dummy.Draw_pencil@1134affc
Square Drawing using pencil
dummy.Draw_pen@6bdf28bb
Square Drawing using pen
```

Abstract class example: With non-abstract method

```
abstract class shapes{
    abstract void square();
    abstract void rectangle();
    void circle() {
        System.out.println("Draw circle");
    }
}

class Draw_pencil extends shapes{
    void square(){
        System.out.println
        ("Square Drawing using pencil");
    }
    void rectangle() {
        System.out.println
        ("Rectangle Drawing using pencil");
    }
}
```

```
public class Oops {
    public static void
    main(String[]arg) {
        shapes obj1 = new
        Draw_pencil();
        System.out.println(obj1);
        obj1.square();
        obj1.circle();
    }
}
```

OP:

```
dummy.Draw_pencil@1134affc
Square Drawing using pencil
Draw circle
```

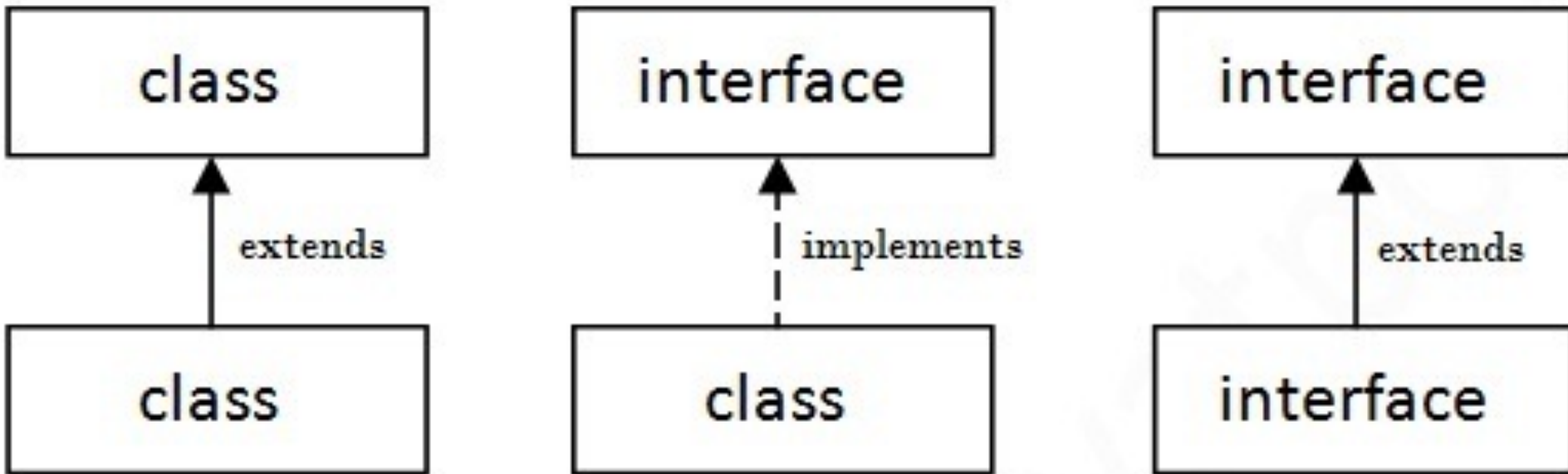
Interface:

- The interface in java is a mechanism to achieve abstraction
- There can be only abstract methods in the java interface not method body
- It is used to achieve abstraction and multiple inheritance in Java

Use of Interface:

- It is used to achieve abstraction
- By interface, we can support the functionality of multiple inheritance

Relationship between classes and interfaces:



Interface class example

```
interface shapes{
    void square();    //No need of
    void rectangle(); //abstract keyword
    static void circle() {
        System.out.println("Draw circle");
    }
    default void triangle() {
        System.out.println("Draw Triangle");
    }
}

class Draw_pencil implements shapes{
    public void square(){
        System.out.println("Square Drawing using
pencil");
    }
    public void rectangle() {
        System.out.println("Rectangle Drawing using
pencil");
    }
}
```

```
public class Oops {
    public static void
main(String[]arg) {
    shapes obj1 = new
Draw_pencil();
    System.out.println(obj1);
    obj1.square();
    shapes.circle();
}
}
```

OP:

```
dummy.Draw_pencil@1134affc
Square Drawing using pencil
Draw circle
```

Interface class example

```
interface area{
    final double pi = 3.14;
    double cal(double a,double b);
}

class rectangle implements area{
    public double cal(double
s1,double s2) {
    return s1*s2;
}
}

class circle implements area{
    public double cal(double
r1,double r2) {
    return pi*r1*r2;
}}
```

```
public class Oops {
    public static void
main(String[]arg) {
    area r=new rectangle();
    area c=new circle();
    System.out.println(r.cal(10,20
));
    System.out.println(c.cal(2,
2));
}
}
```

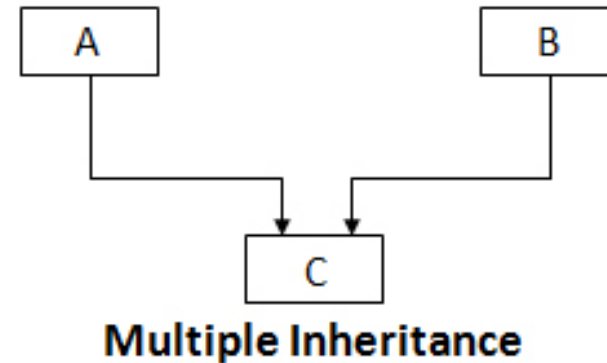
OP:
200.0
12.56

Why multiple inheritance is not supported in java?

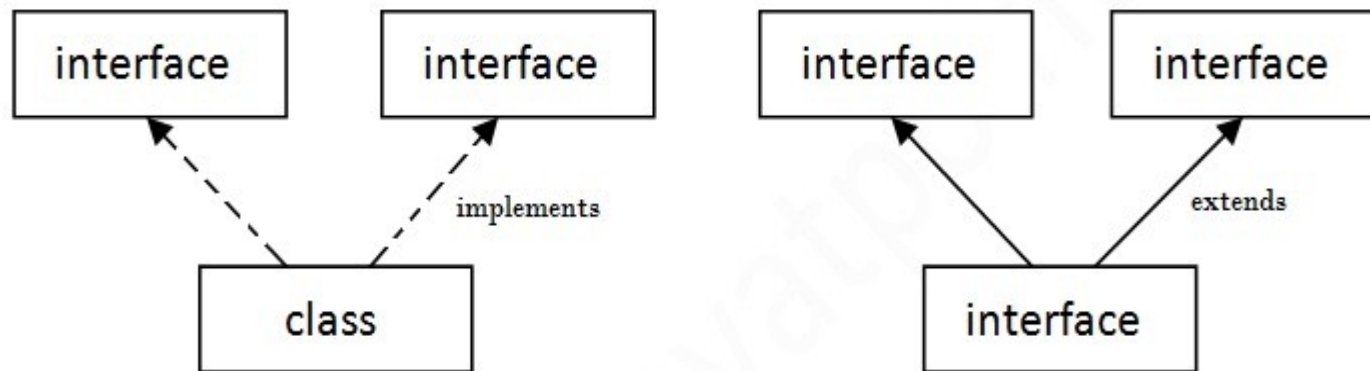
```
class A{
void msg(){System.out.println("Hello");}
}

class B{
void msg(){System.out.println("Welcome");}
}

class C extends A,B
{
    Public Static void main(String args[]){
        C obj=new C();
        obj.msg();
    }
}
```



- To reduce the complexity and simplify the language, multiple inheritance is not supported in java.
- Since compile time errors are better than runtime errors, java renders compile time error if you inherit 2 classes.
- Multiple inheritance can be achieved with the help of interface.



Multiple Inheritance in Java

Differences between abstract class and Interface :

Abstract class	Interface
1) Abstract class can have abstract and non- abstract methods.	Interface can have only abstract methods.
2) Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.

Encapsulation:

- It's a process of wrapping code and data together into a single unit
- We can create a fully encapsulated class in java by making all the data members of the class private. Now we can use setter and getter
- By providing only setter or getter method, you can make the class **read-only or write-only** methods to set and get the data in it
- It provides you the **control over the data**

Access Modifiers:

- **Private** – accessible only within class
- **Default** – accessible only within package
- **Protected** – accessible within package and outside the package but through inheritance only.
 - ✓ The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.
- **Public** – accessible everywhere. It has the widest scope among all other modifiers

Private access modifier:

```
package dummy;  
import java.util.*;  
class Terv{  
    //Visible only with in class  
    private int fact(int n) {  
        if(n==1) {  
            return 1;  
        }  
        else return n*fact(n-1);  
    }  
}
```

Public access modifier:

//Visible everywhere

```
public int fact_get(int n) {  
    int res = fact(n);  
    return res;  
}
```

Protected access modifier:

//Visible Only to inherited class

```
protected static int fib(int n) {  
    if(n==0) return 0;  
    if(n==1) return 1;  
    return fib(n-1)+fib(n-2);  
}  
}
```

Public access modifier:

```
//Public class only get visible to other  
packages
```

```
public class info extends Terv{  
public static void main(String []arg) {  
Terv obj = new Terv();  
System.out.println(obj.fact_get(10));  
}//End of main  
}//End of class
```

Imported to another package:

```
package ecelab;
import dummy.info;
public class TeamTerv extends info{
public static void main(String[] args)
{
info obj = new info();
int res = obj.fact_get(5);
System.out.println(res);
int res2 = info.fib(8);
System.err.println(res2);
}
}
```